

Navegação Autónoma na Formula Student: Uma Abordagem Baseada em Aprendizagem por Reforço

Luís Miguel Pereira Silva¹ and Pedro Miguel S. A. Urbano dos Reis¹

Mestrado em Inteligência Artificial, Universidade do Minho, Braga, Portugal
{pg60390,pg59908}@alunos.uminho.pt

Abstract. Este documento apresenta o planeamento de um trabalho prático centrado no desenvolvimento de um sistema de condução autónoma para a competição Formula Student, recorrendo a técnicas de Aprendizagem por Reforço (RL). O problema consiste em treinar um agente capaz de navegar numa pista delimitada por cones coloridos, respeitando as regras da categoria *Driverless* (FS-AI). Será desenvolvido um ambiente de simulação 2D com modelo cinemático de bicicleta, geração procedural de pistas e penalizações realistas. A metodologia assenta na implementação de um agente *Soft Actor-Critic* (SAC) para controlo em espaços de ação contínuos, utilizando *Proximal Policy Optimization* (PPO) como *baseline* comparativo.

Keywords: Aprendizagem por Reforço · Condução Autónoma · Formula Student · Soft Actor-Critic · Simulação

1 Introdução e Definição do Problema

A competição Formula Student (FS) é a maior competição de engenharia automóvel universitária a nível mundial. A categoria *Driverless* (FS-AI) desafia equipas a desenvolver veículos totalmente autónomos capazes de navegar numa pista delimitada por cones coloridos — azuis (esquerda), amarelos (direita) e laranja (partida/chegada) — sem qualquer intervenção humana [1]. O veículo deve detetar cones, planejar trajetórias e executar ações de direção e aceleração em tempo real.

A solução proposta assenta em Aprendizagem por Reforço (RL) [2]: treinar um agente que aprenda a conduzir maximizando o progresso e minimizando penalizações. Este relatório documenta a metodologia planeada, incluindo ambiente, algoritmos, ferramentas e formas de validação. O código encontra-se em: <https://github.com/pedroreis2468/AR>.

2 Ambiente de Simulação e Geração de Dados

O treino será conduzido inteiramente em simulação, com um ambiente desenvolvido de raiz seguindo a API *Gymnasium*.

Modelo do veículo. O veículo é modelado com um modelo cinemático de bicicleta referenciado ao centro de gravidade, calibrado para um FS típico (~ 250 kg, *wheelbase* de 1.55 m): $\dot{x} = v \cos(\theta + \beta)$, $\dot{y} = v \sin(\theta + \beta)$, onde $\beta = \arctan(l_r \tan \delta / L)$ é o ângulo de deslizamento lateral. O modelo inclui limite de aceleração combinada ao atrito disponível ($\mu \cdot g$).

Geração procedural e percepção. As pistas são geradas a partir de pontos de controlo numa elipse perturbada, suavizados com *splines* cúbicas periódicas, randomizando a cada episódio. A percepção é simulada por um sensor de cones (FOV: 180° , alcance: 15 m), que retorna os 3 cones mais próximos de cada lado com ruído gaussiano ($\sim 5\%$).

Regras FS-AI. Cones derrubados: +2 s por cone (removidos da percepção, sem terminação). Cones laranja: +4 s. Terminação (DOO) apenas se ≥ 10 cones derrubados, desvio > 5 m do *centerline*, ou *off-course* > 2 s consecutivos.

3 Metodologia e Algoritmos

O problema é formulado como um MDP com $\gamma = 0.99$:

- **Observação** ($\mathbf{s} \in \mathbb{R}^{20}$): estado do veículo (5 dims), posições relativas dos cones (12 dims), distância às fronteiras e erro de *heading* (3 dims).
- **Ação** ($\mathbf{a} \in [-1, 1]^2$): direção normalizada e aceleração.
- **Recompensa**: $r_t = w_p \cdot \Delta_{\text{prog}} - w_s \cdot |\Delta \delta| - w_l \cdot d_{\text{lat}}^2 - w_t \cdot \Delta t - P_{\text{cones}}$, combinando progresso, suavidade, penalidade lateral, custo temporal e penalizações por cones. Os pesos w_i serão ajustados experimentalmente.

O algoritmo principal será o **Soft Actor-Critic (SAC)** [3], um método *off-policy* com maximização de entropia, implementado com *Gaussian actor*, *twin Q-networks* e temperatura α auto-ajustada. Como *baseline*, será utilizado o **PPO** (*on-policy*) via Stable-Baselines3. Aplicar-se-á **domain randomization** [4] variando massa ($\pm 15\%$) e atrito ($\pm 20\%$).

4 Ferramentas e Arquitetura de Software

O desenvolvimento assenta em Python (≥ 3.10) com: **Gymnasium** ≥ 0.29 (API do ambiente), **PyTorch** ≥ 2.0 (redes *Actor-Critic*), **Stable-Baselines3** ≥ 2.1 (PPO *baseline*), **PyGame** ≥ 2.5 (renderização 2D), **TensorBoard** (monitorização) e **Git+GitHub** (versionamento e entrega).

5 Protocolo de Validação

A validação estrutura-se em três vertentes: (1) **Desempenho desportivo** — cada agente avaliado em 20 pistas nunca vistas, reportando taxa de voltas completas, tempo corrigido (bruto + penalizações FS-AI), cones derrubados e velocidade média (média $\pm$ desvio-padrão); (2) **Eficiência de aprendizagem** — monitorização ao longo de 500k *steps* (SAC) e 1M *steps* (PPO) da *reward* cumulativa, *episode length* e *Q-value* estimado; (3) **Comparação SAC vs. PPO**

Table 1. Planeamento e datas de entrega.

Período	Tarefa
Até 30 abr	Entrega do relatório de planeamento
1–18 mai	Treino intensivo (SAC + PPO) e avaliação de métricas
19–25 mai	Experiências de <i>domain randomization</i>
26 mai–1 jun	Preparação da apresentação
2 jun	Apresentação e resultados preliminares
3–15 jun	Incorporação de feedback; redação do relatório final
16 jun	Entrega do relatório final (LNCS)

— contraste direto em desempenho final, *sample efficiency*, tempo de treino e impacto do *domain randomization* na generalização.

6 Cronograma

References

1. Kabzan, J., et al.: AMZ Driverless: The full autonomous racing system. *Journal of Field Robotics* **37**(7), 1267–1294 (2019)
2. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press, Cambridge, MA (2018)
3. Haarnoja, T., et al.: Soft Actor-Critic algorithms and applications. *arXiv preprint arXiv:1812.05905* (2018)
4. Ulrich, F., Wehrli, T.: End-to-End deep reinforcement learning for autonomous racing dynamics. Bachelor Thesis, ZHAW School of Engineering (2024)